Árboles

Objetivos:

- Presentar la estructura no lineal más importante en computación
- Mostrar la especificación e implementación de varios tipos de árboles
- Algoritmos de manipulación de árboles

Contenido

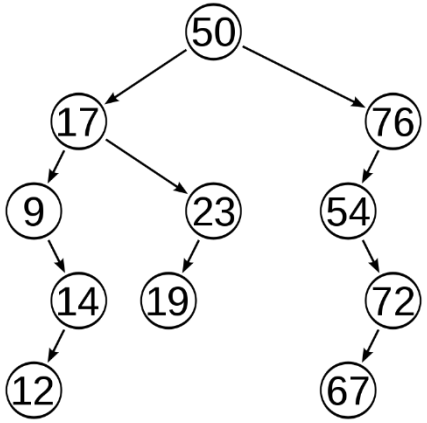
1. Introducción
    1.1. Definición
    1.2. Terminología
    1.3. Recorridos
2. Árboles binarios
    2.1. Definición
    2.2. Especificación
    2.3. Implementación
3. Heap o Montículo binario
    3.1. Definición
    3.2. Especificación
    3.3. Implementación
4. Árboles binarios de búsqueda
    4.1. Definición
    4.2. Especificación
5. Árboles binarios equilibrados
    5.1. Árboles AVL
6. Árboles generales
    6.1. Especificación

Árbol Binario Equilibrado

- Los **árboles binarios de búsqueda** son una estructura sobre la cual se pueden realizar eficientemente las operaciones de búsqueda, inserción y eliminación
  - La eficiencia de las operaciones depende exclusivamente de la altura del árbol
  - Árbol de *N* nodos perfectamente equilibrado $\Rightarrow$ el coste de acceso es de orden logarítmico: O(log *N*)

Árbol Binario Equilibrado

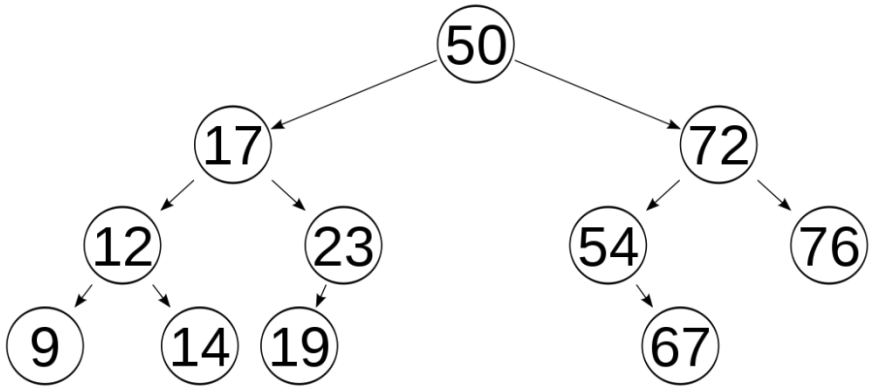
- Sin embargo, si el árbol crece o decrece descontroladamente, el rendimiento puede disminuir considerablemente



  - Caso más desfavorable: insertar un conjunto de claves ordenadas en forma ascendente o descendente
  - Coste de acceso: O(N)
- **Árboles equilibrados**:
  - Aseguran un coste logarítmico sin exigir que el árbol sea completo
  - Idea central: realizar acomodos o equilibrios después de inserciones o eliminaciones de elementos

Árbol AVL

- Un **árbol AVL** es un árbol binario de búsqueda en el que para todo nodo A del árbol la altura de los subárboles izquierdo y derecho no debe diferir en más de una unidad.
- La condición de equilibrio asegura una profundidad del árbol de O(log *N*).

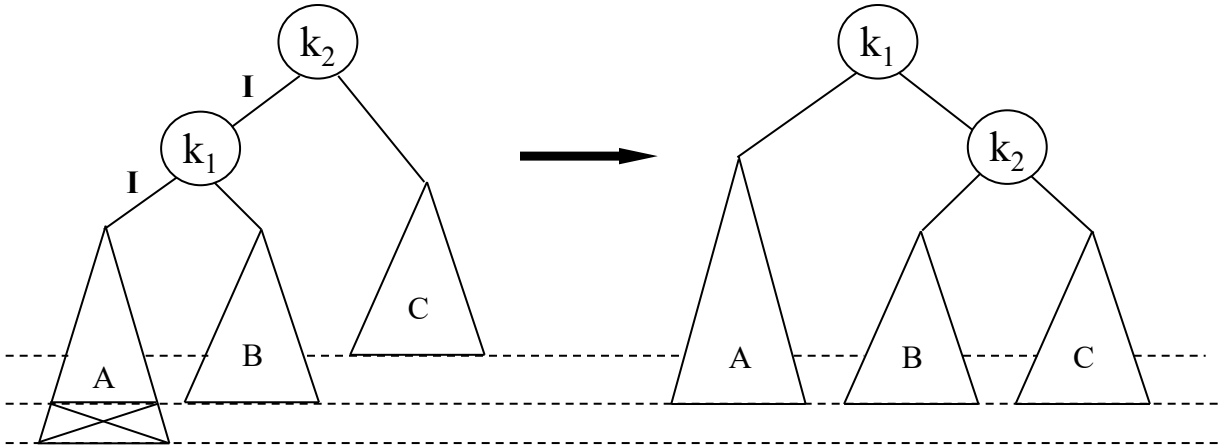


Árbol AVL

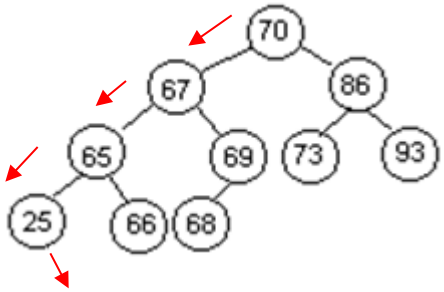
- **Factor de equilibrio** (FE) de un nodo: diferencia entre la altura de los subárboles. Los valores que puede tomar son -1, 0, 1. Si llegara a tomar los valores -2 o 2 $\Rightarrow$ debe reestructurarse el árbol
  - Reestructurar el árbol significa rotar los nodos del mismo
    - Rotación simple: involucra dos nodos
      - Por la rama izquierda
      - Por la rama derecha
    - Rotación compuesta: afecta a tres nodos
      - Por las ramas izquierda y derecha
      - Por las ramas derecha e izquierda

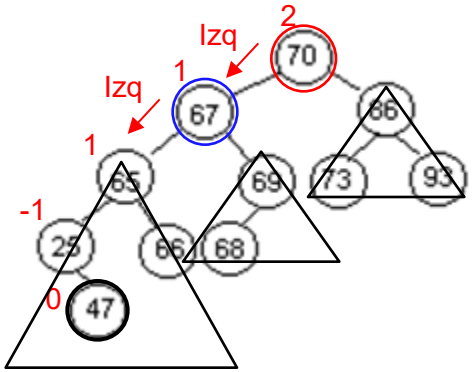
Rotación simple I-I

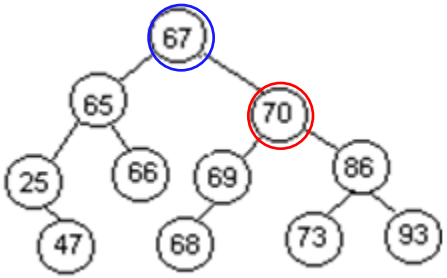
- **Rotación simple (I-I)**



- Ejemplo: Inserción del nodo 47 en un árbol AVL

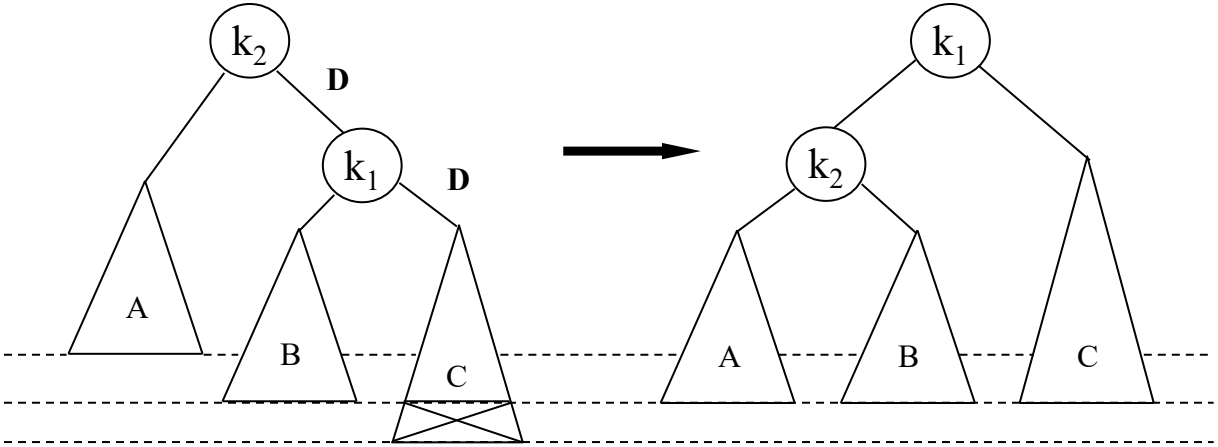




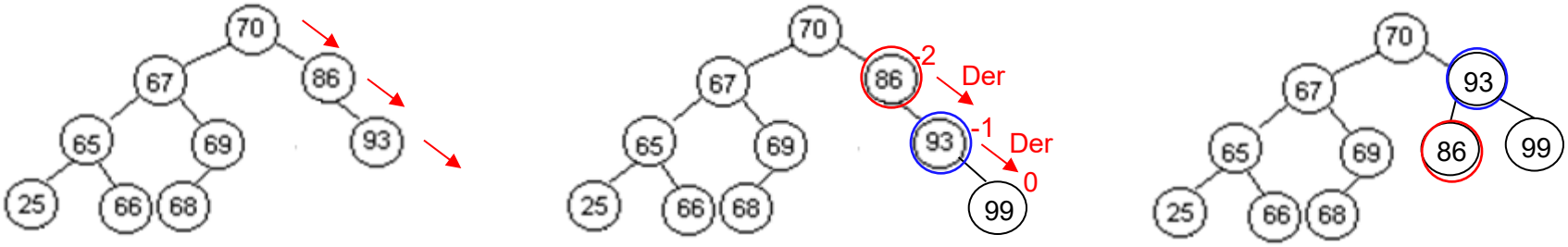


Rotación simple D-D

- **Rotación simple (D-D)**

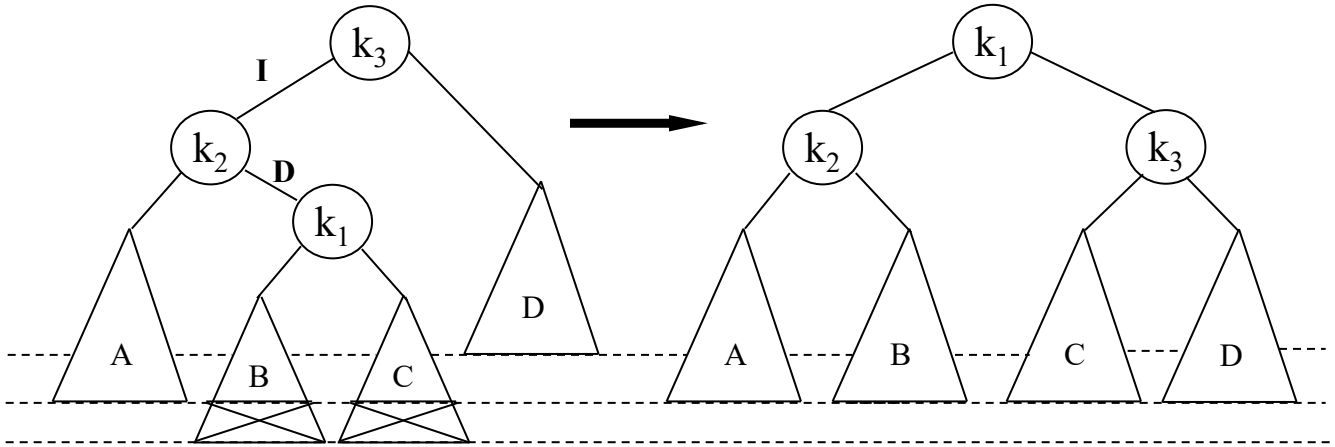


- Ejemplo: Inserción del nodo 99 en un árbol AVL

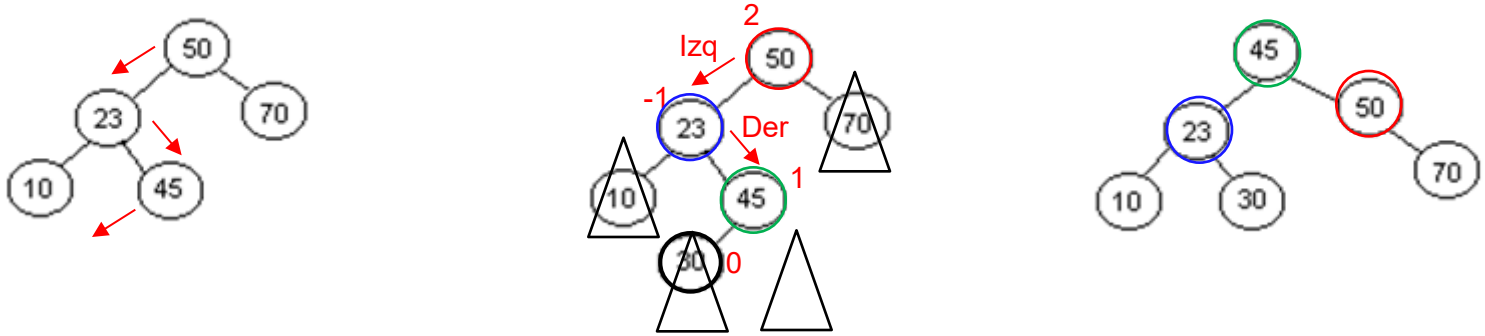


Rotación compuesta I-D

- **Rotación compuesta (I-D)**

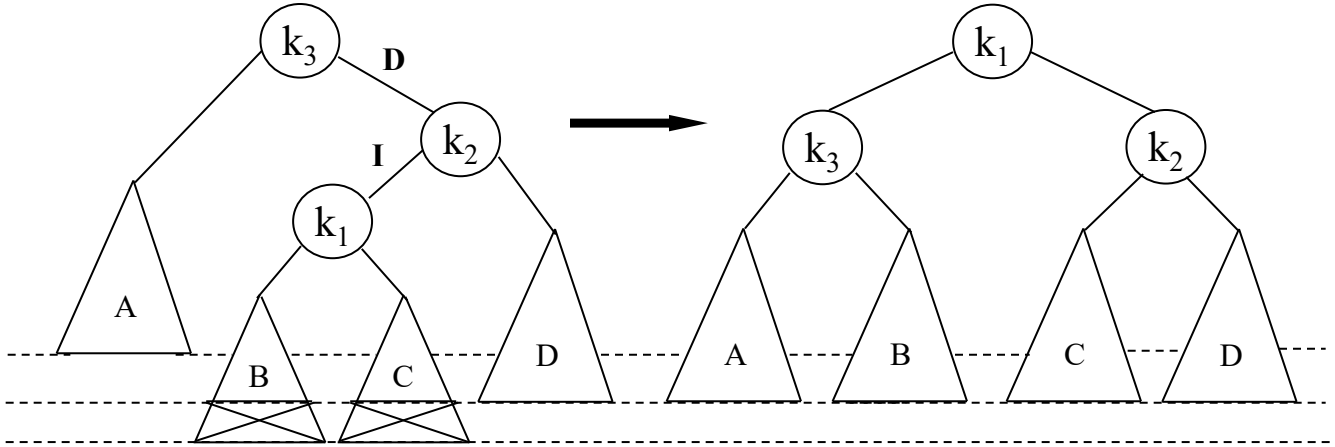


- Ejemplo: Inserción del nodo 30 en un árbol equilibrado

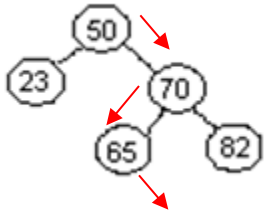


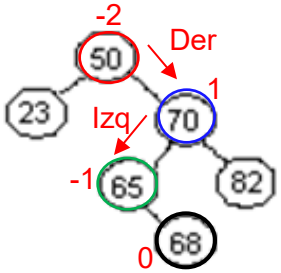
Rotación compuesta D-I

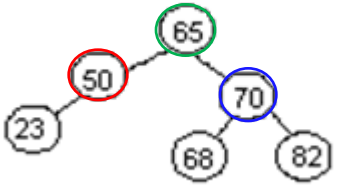
- **Rotación compuesta (D-I)**



- Ejemplo: Inserción del nodo 68 en un árbol equilibrado







Insertar

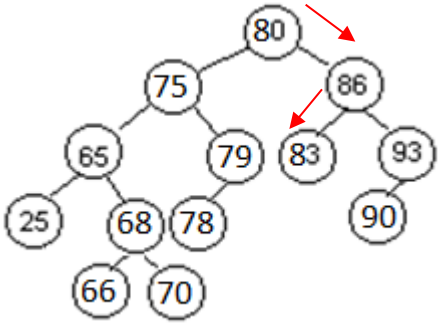
- **Resumen operación insertar:**
  - Insertar el nodo como en un árbol binario de búsqueda
  - En el regreso por el camino de inserción se comprueba el FE de los nodos
  - Si un nodo presenta un FE incorrecto (2 o –2) se detiene el retroceso en este punto, se baja dos niveles por la rama más larga y se reestructura el árbol
  - Una inserción provoca una **única** reestructuración

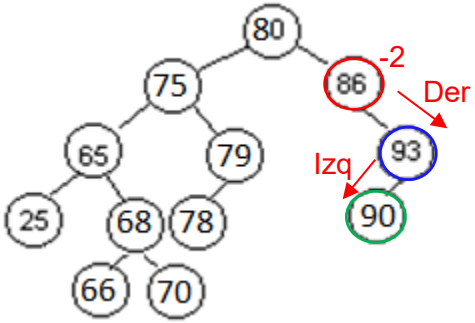
Suprimir

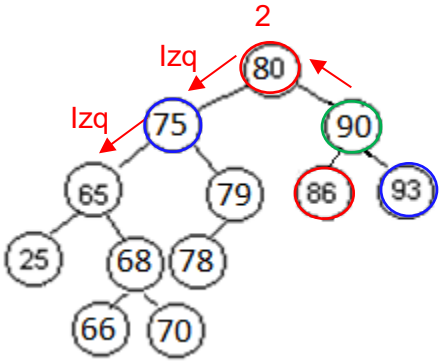
- **Resumen operación suprimir:**
  - Suprimir el nodo como en un árbol binario de búsqueda
  - En el regreso por el camino de supresión se comprueba el FE de los nodos
  - Si un nodo presenta un FE incorrecto (2 o –2) se reestructura el árbol y **se continua el ascenso hasta llegar a la raíz**
  - Una supresión puede provocar **varias** reestructuraciones

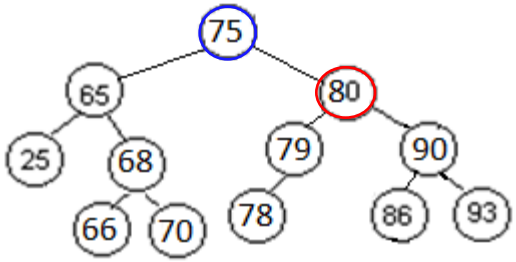
Ejemplo

- Ejemplo: eliminar el nodo 83









Ejercicios

Ejercicios de autoevaluación:

- Dibuja el árbol AVL que resulta después de insertar los elementos: 10, 33, 58, 40, 75, 49, 7, 18, 25, 15, 36, 3.
- Dibuja el árbol AVL que resulta después de eliminar los siguientes elementos del árbol anterior: 40, 75, 10, 49, 18, 33, 25, 58.
- Dibuja el árbol AVL que resulta después de insertar los elementos: 3, 5, 4, 1, 9, 2, 6 y 7.
- Dibuja el árbol AVL que resulta después de eliminar los siguientes elementos del árbol anterior: 2, 3, 6, 7.